

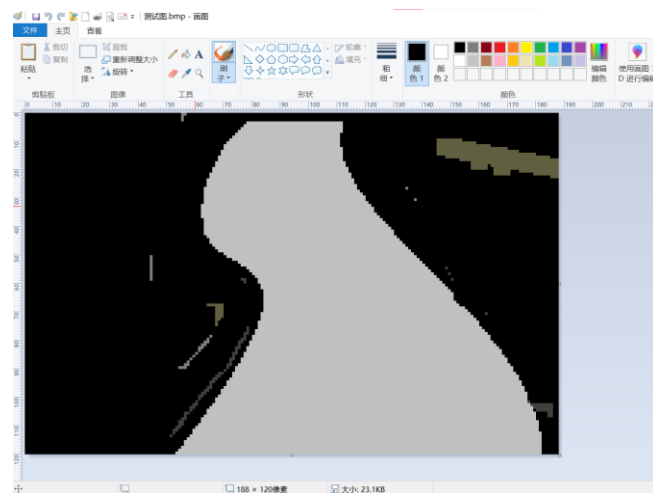
Tip: 感觉这也不能叫做开源, 只能是一些分享吧。

最近想起了之前写的程序, 虽然都是些不成熟的想法, 并且已经不打算继续探索下去了, 但是总感觉这也是一种思路, 所以就分享出来吧, 说不定有用呢。

不善言辞, 进入正题:

智能车图像处理可视化以及灰度图八邻域的一些想法分享

因为之前的图都被我删掉了, 所以只能自己画一个了, 想着在网上找一个, 但是没找到, 所以将就看吧, 不过不重要, 有摄像头可以自己采样嘛, 下面我就都用这张图演示了, 尽量讲的细一些。 **Tip: 注意我这里使用的图片为考虑寻到边界的情况, 所以我把图像顶部抹黑了, 注意注意!**



在此之前, 还需要做一些准备工作, 配置一下 OpenCV C++ 环境, 可以实现视频呈现的可视化效果。

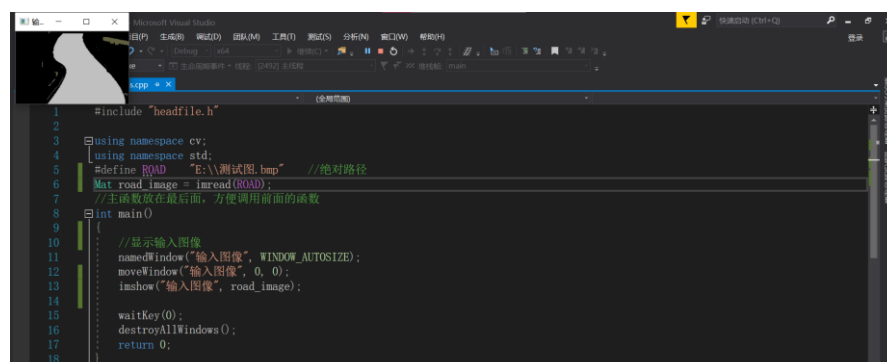
VisualStudio2019安装配置OpenCV的C++环境

文章目录文章目录前言环境引用VS2019下载与安装OpenCV下载与安装下载安装创建&配置C++项目测试 前言 自己近期在学习机器学习, 想要动手做一个人脸识别项目, 而OpenCV是一个非常不错的开源计算机视觉和机器学习库...

配置环境这一步, 稍微有点繁琐, 可能需要多次配置, 这需要耐心, 我自己之前也配置了很多次吧, 多查教程 (如上图)。

Tip: 只要能实现从自己的文件夹里读出任意图片并显示, 就说明配置已经完成, 可以进行下一步了。

接下来进入下一步, 把赛道图片读进来显示:

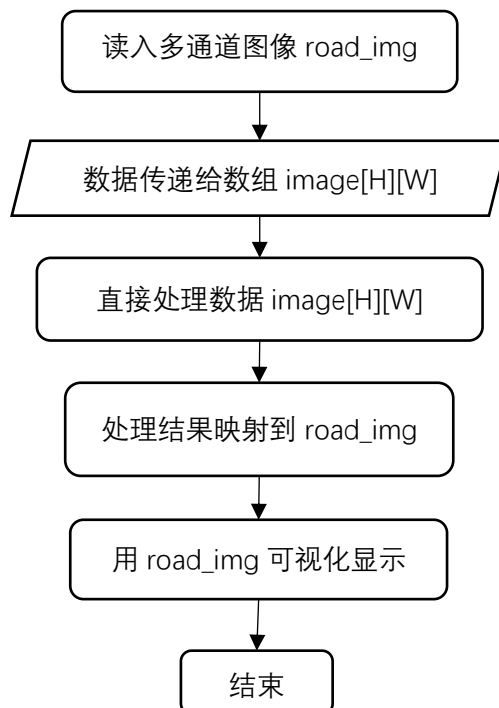


显示成功，但是现在还不能使用，因为我们希望在此处编写的各种处理程序可以直接应用于单片机，所以我们要做一下数据传递。

```
//定义一个二维数组，或者一维数组，这里我就用二维数组了
/*图像数组变量声明*/
uint8 image[image_h][image_w]; //单片机需要用到的数组
uint8 bin_image[image_h][image_w]; //单片机需要用到的数组
void Data_trans(Mat image)
{
    //这里就是传参，把三通道数组转换成单通道，好像有个API可以直接转换，只是我没用它
    #define use_num 1 //1就是不压缩，2就是压缩一倍
    uint16 i = 0, j = 0, row = 0, line = 0;
    int channels = image_.channels();
    for (i = 0; i < image_h; i += use_num) //
    {
        for (j = 0; j < image_w; j += use_num) //
        {
            image[row][line] = image_.at<uchar>(i, j*channels); line++;
        }
        line = 0; row++;
    }
}
```

很简单，就是一个 for 循环，目的就是把多通道数据转换成单通道的，后续我们绘图可视化依旧是使用的原来的多通道图像。

接下来用流程图来梳理一下这个过程。



按照这一套流程下来，我们针对 `image[image_h][image_w]` 进行的所有处理都能直接用于单片机，换句话说，在 OpenCV 写好了复制粘贴过去，啥都不用改。

过程清楚了，那么用大名鼎鼎的大津法二值化来实践一下吧：

```
int main()
{
    //显示输入图像
    uint8 image_threshold;
    namedWindow("输入图像", WINDOW_AUTOSIZE);
    moveWindow("输入图像", 0, 0);
    imshow("输入图像", road_image);
    Data_trans(road_image); //传递参数，方便处理
    image_threshold = otsuThreshold(image[0], image_h, image_w);
    waitKey(0);
    destroyAllWindows();
    return 0;
}
```

定义阈值

传递参数

计算阈值

这里的 otsuThreshold 函数是我在网上抄的，部分程序如下，之前我[开源的八邻域](#)（[链接摁住 ctrl 点击观看](#)）里面有一样的程序，自行参考哦：

```
uint8 otsuThreshold(uint8 *image, uint16 col, uint16 row)
{
#define GrayScale 256
    uint16 Image_Width = col;
    uint16 Image_Height = row;
    int X; uint16 Y;
    uint8 *data = image;
    int HistGram[GrayScale] = { 0 };
    uint32 Amount = 0;
    uint32 PixelBack = 0;
    uint32 PixelIntegralBack = 0;
    uint32 PixelIntegral = 0;
    int32 PixelIntegralFore = 0;
    int32 PixelFore = 0;
    double OmegaBack = 0, OmegaFore = 0, MicroBack = 0, MicroFore = 0, SigmaB = 0,
    Sigma = 0; // 类间方差;
    uint8 MinValue = 0, MaxValue = 0;
```

二值化的过程就和我们正常的处理方式一样了，大于阈值设定为白色，小于阈值设定为黑色，简单易懂。

```
{
    //显示输入图像
    uint8 image_threshold;
    namedWindow("输入图像", WINDOW_AUTOSIZE);
    moveWindow("输入图像", 0, 0);
    imshow("输入图像", road_image);
    Data_trans(road_image); //传递参数, 方便处理
    image_threshold = otsuThreshold(image[0], image_h, image_w);
    //二值化
    uint8 i, j;
    for (i = 0; i < image_h; i++)
    {
        for (j = 0; j < image_w; j++)
        {
            if (image[i][j] > image_threshold) bin_image[i][j] = 255;
            else bin_image[i][j] = 0;
        }
    }
}
```

然后就是随后一步了，可视化映射了，这里多说两句，其实叫做映射理解起来可能有点抽象了，我们不如这样理解：

如果 `image[0][0]` 的值是 255，那么意味着左上角第一个像素点是白色，我们在显示的时候就需要把第一个点设置为白色，那么怎么做呢，如果直接把值传递给 `roadimg<0,0>` 能不能成功呢？答案是否，因为我们最开始输入的图像是多通道的图，它有三个通道的数据，直接赋值只是给其中一个通道赋值，所以要将 `(0,0)` 这个位置的像素点设置成白色，就需要把输入图像的三个通道全部赋值为 255。

原理理解了，实操如下所示：

```

//映射输出图像
for (uint8 h = 0; h < image_h - 1; h++)
{
    for (uint8 w = 0; w < image_w - 1; w++)
    {
        if (bin_image[h][w] == 255)
        {
            road_image.at<Vec3b>(h, w)[0] = 255; //三通道赋值
            road_image.at<Vec3b>(h, w)[1] = 255;
            road_image.at<Vec3b>(h, w)[2] = 255;
        }else
        {
            road_image.at<Vec3b>(h, w)[0] = 0;
            road_image.at<Vec3b>(h, w)[1] = 0;
            road_image.at<Vec3b>(h, w)[2] = 0;
        }
    }
}

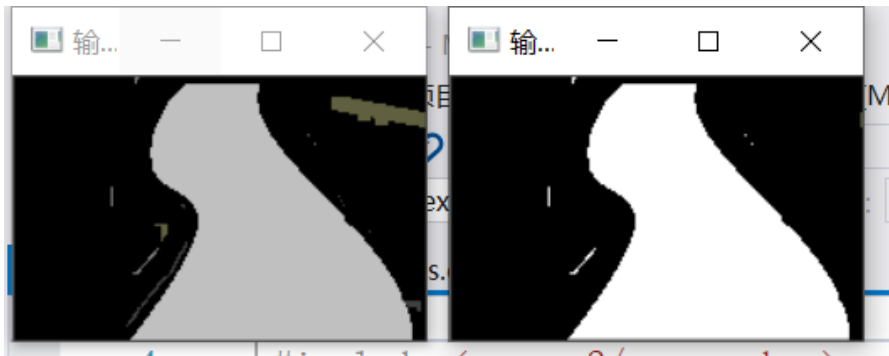
```

最后再显示一次 roadimage 就行了。

```

namedWindow("输出图像", WINDOW_AUTOSIZE);
moveWindow("输出图像", 200, 0);
imshow("输出图像", road_image);

```



至此，可视化已经完成，只要按照流程图所示操作图像，就可以在 OpenCV 上编写好处理函数，然后运行看效果，就可以不用频繁的趴到赛道上看结果了。

后续什么画点，划线等操作都是基操了，这里不多提，我这次给出的程序里有，一看就能懂。

一些灰度八邻域局部阈值的想法：

之前参考了一下上交的自适应阈值，应用在八邻域上面试了一下，效果呢，不尽人意，可能八邻域的劣势就在这里吧，这种方式用在上交的迷宫法上面倒是可以稳定寻出边界，因为它不用判断边界外的像素点，八邻域每次都要比较边界外的点，很容易卡在一个地方死循环，不稳定。

对于八邻域来说，如果它的下一个点走到了周围基本都是较暗的的点的位置，那么它求出来的平均阈值就会很小，选取到的下一次的循环的中心点大概率是个黑色，于是乎死循环就开始了。为了让其能够以局部阈值的方式用较小的计算量找到尽量靠近白色点的区域寻找边界，我在求平均阈值的同时，手动给该阈值加上了几个白色点的值，然后再求一次平均值，那么该区域求出来的平均阈值靠近白色点的概率就被大大提高了。如下图程序所示：

```

//左边, 右边同理
for (i = 0; i < 8; i++)//传递8F坐标
{
    search_filds_l[i][0] = center_point_l[0] + seeds_l[i][0];//x
    search_filds_l[i][1] = center_point_l[1] + seeds_l[i][1];//y
    thres_l = 0;
    for (uint8 j = 0; j < 25; j++)//统计周围5*5范围内的像素值
    {
        thres_l += image[search_filds_l[i][1] + square_5[j][1]][search_filds_l[i][0] + square_5[j][0]];
    }
    thres_l = thres_l + 201 + 200 + 200 + 201 + 200 + 199 + 196 + 194 + 192; //加入9个
    白点的值

    thres_l /= 33;//计算局部平均阈值
    if (image[search_filds_l[i][1]][search_filds_l[i][0]] > thres_l)
    {
        bin_image[search_filds_l[i][1]][search_filds_l[i][0]] = 255;
    }
    else bin_image[search_filds_l[i][1]][search_filds_l[i][0]] = 0;
}

```

具体呈现的对比效果嘛，可以参考我之前发布的视频（摁住 ctrl 键点击链接即可观看）：

[智能车图像处理进阶——灰度八邻域: 解决一个小问题, 感觉可以继续写下去, 大佬轻点喷。](#)
[哔哩哔哩 bilibili](#)

这里说两句吧：

为什么一定要用 OpenCV 呢？其实不是一定要用，每个人都有自己的学习方式，当然也不是每个人都能用上功能强大的上位机，我选择 OpenCV 来进行可视化操作的原因也很简单，因为我只做图像处理，其他的控制全然不考虑的情况下，OpenCV 作为一款强大的开源工具，必然是最佳的选择。

当然，分享这样的方式，其实并没有太多的用到 OpenCV 的东西，就是一些显示罢了。鄙人作为机械专业的学生，其实并没有太多的接触到这些东西，只是浅浅的运用一下，参加这个比赛，图像处理对大多数人来说是一件很恶心的事情，因为太多恶心的情况要考虑了，要做出优秀的控制响应，需要得到的误差值必须同样优秀，所以花在这上面的调试时间也会相对较多，这是一件很考验耐心的事情。虽然八邻域计算确实快，但是稳定性依旧有待考证，我之前开源的八邻域也是做了各种条件限制的情况下才有了基本稳定运行的效果。

之前有很多人移植八邻域说卡死，运行不流畅，大概率是你在移植的时候删减了我加上去的某些东西，还有就是图像大小一定要对应，我后来帮助一些人移植过了也有好几次吧，除了有个老哥用的自建库以外，其他的我没有移植失败过，无一例外都很流畅，所以移植的时候可以先直接复制粘贴，改改数据类型定义就可以运行了，成功了之后再根据自己的需求增删改查，找对合适的方法才能起步更快。

如果你在 b 战上问我为什么卡，我大多数情况都没有回复，因为这个时候我大概率在上班，在 emo，在斤斤计较生活中一切让人不爽的瞬间，你这个突如其来的问题其实我也一头雾水啊，所以回复你的概率太低了，真的见谅。



尬笑

希望大佬们遇到问题，尽量多思考，我当时写这份程序的时候也不是一次性就写完的，也是偶尔休息个一两天，思考出的解决方法，**所以，不要轻言放弃，要开动脑筋多多思考，因为念念不忘，才必有回响。**



最后，如果觉得我的分享对你有帮助的话，一定要投币啊，拜托拜托。

拜托拜托

